

网络聊天室服务端开发

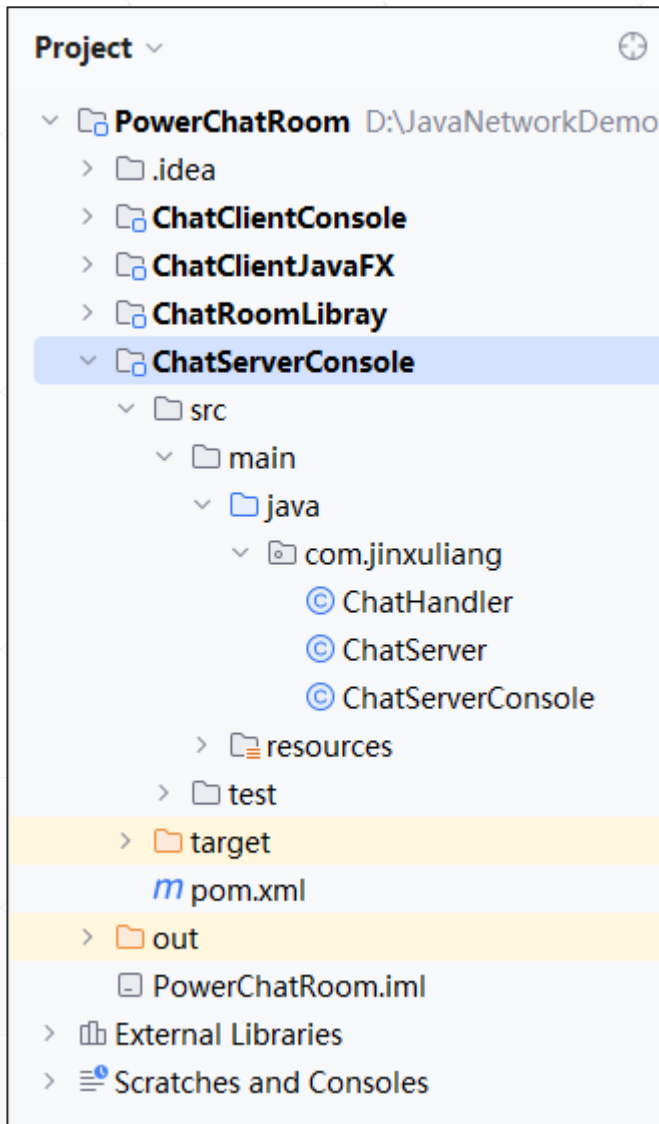
北京理工大学计算机学院
金旭亮

项目成员

ChatServerConsole.java: 程序入口点

ChatServer.java: 封装聊天的主要功能

ChatHandler.java: 当用户连接请求进来之后，此类负责响应，它会调用ChatServer封装的相应功能



程序入口点

```
public class ChatServerConsole {  
  
    public static void main(String[] args) {  
        ChatServer chatServer = new ChatServer();  
        chatServer.start();  
    }  
}
```

ChatServer-信息存储

```
public class ChatServer {  
  
    //用于监听客户端连接的端口  
    4 usages  
    private ServerSocket serverSocket;  
    //用于保存在线用户信息  
    8 usages  
    private Map<InetSocketAddress, Writer> connectedClients;  
  
    1 usage  
    public ChatServer() { connectedClients = new HashMap<>(); }  
}
```

ChatServer-添加客户端

//有客户端连接进来之后，将其加入到在线用户清单中，同时在控制台窗口输出一条信息

1 usage

```
public synchronized void addClient(Socket socket) throws IOException {  
    if (socket != null) {  
        int port = socket.getPort();  
        var ip = socket.getInetAddress();  
        InetSocketAddress socketAddress = new InetSocketAddress(ip, port);  
        BufferedWriter writer = new BufferedWriter(  
            new OutputStreamWriter(socket.getOutputStream())  
        );  
        connectedClients.put(socketAddress, writer);  
        socketAddress = new InetSocketAddress(ip, port);  
        System.out.println("客户端[" + port + "]已连接到服务器");  
    }  
}
```

ChatServer-移除客户端

```
//用户下线
1 usage
public synchronized void removeClient(Socket socket) throws IOException {
    if (socket != null) {
        int port = socket.getPort();
        var ip = socket.getInetAddress();
        InetSocketAddress socketAddress = new InetSocketAddress(ip, port);
        if (connectedClients.containsKey(socketAddress)) {
            connectedClients.get(socketAddress).close();
        }
        var result = connectedClients.remove(socketAddress);
        System.out.println("\n客户端[" + socket.getInetAddress() + ":" +
            + socket.getPort() + "]已断开连接,\n当前在线用户数:" +
            + connectedClients.size());
    }
}
```

//判断用户发来的消息是不是“下线”请求

```
1 usage
public boolean readyToQuit(String msg) { return QUIT.equals(msg); }
```

ChatServer-消息转发

//一个用户发来消息, Server转发消息给其他的在线用户

1 usage

```
public synchronized void forwardMessage(Socket socket, String fwdMsg)
    throws IOException {
    var ip = socket.getInetAddress();
    var port = socket.getPort();
    var socketAddress = new InetSocketAddress(ip, port);
    System.out.println("\n转发消息:" + socketAddress);
    for (var client : connectedClients.keySet()) {
        if (!socketAddress.equals(client)) {
            Writer writer = connectedClients.get(client);
            writer.write(fwdMsg);
            writer.flush();
        }
    }
}
```

ChatServer-关闭聊天服务

//关闭聊天服务

1 usage

```
public synchronized void close() {  
    if (serverSocket != null) {  
        try {  
            serverSocket.close();  
            System.out.println("关闭serverSocket");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```


ChatServer-启动聊天服务

```
public void start() {  
    try {  
        // 绑定监听端口  
        serverSocket = new ServerSocket(Constants.DEFAULT_SERVER_PORT);  
        String myIP = InetAddress.getLocalHost().getHostAddress();  
        System.out.println("ChatServerConsole在 " + myIP + ":"  
            + Constants.DEFAULT_SERVER_PORT + " 监听");  
        while (true) {  
            // 等待客户端连接  
            Socket socket = serverSocket.accept();  
            // 创建ChatHandler线程  
            new Thread(new ChatHandler(server: this, socket)).start();  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    } finally {  
        close();  
    }  
}
```

ChatHandler

```
public class ChatHandler implements Runnable {  
    5 usages  
    private ChatServer server;  
    8 usages  
    private Socket socket;  
  
    1 usage  
    public ChatHandler(ChatServer server, Socket socket) {  
        this.server = server;  
        this.socket = socket;  
    }  
}
```

注入一个ChatServer，调用它的相关方法来完成聊天流程.....

ChatHandler-实现聊天流程

```
@Override
public void run() {
    if (socket.isInputShutdown() ||
        socket.isOutputShutdown() || socket.isClosed()) {
        return;
    }
    try {
        // 存储新上线用户
        server.addClient(socket);
        // 读取用户发送的消息
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );
        String msg = null;
```

```
while ((msg = reader.readLine()) != null) {
    ChatMessage clientSendMessage = null;
    clientSendMessage = ChatMessage.fromJson(msg);
    if (server.readyToQuit(clientSendMessage.getMessage())) {
        clientSendMessage = ChatMessage.builder()
            .userName(clientSendMessage.getUserName())
            .message("用户已经退出")
            .build();
    }
    System.out.println(clientSendMessage.getUserName() + ":"
        + clientSendMessage.getMessage());
    // 将消息转发给聊天室里在线的其他用户
    server.forwardMessage(socket, fwdMsg: ChatMessage.toJson(clientSendMessage) + "\n");
}
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        server.removeClient(socket);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
```